夜の学校から脱出

第5回からのグループ開発
共通ベースを読み、5人で機能を実装する

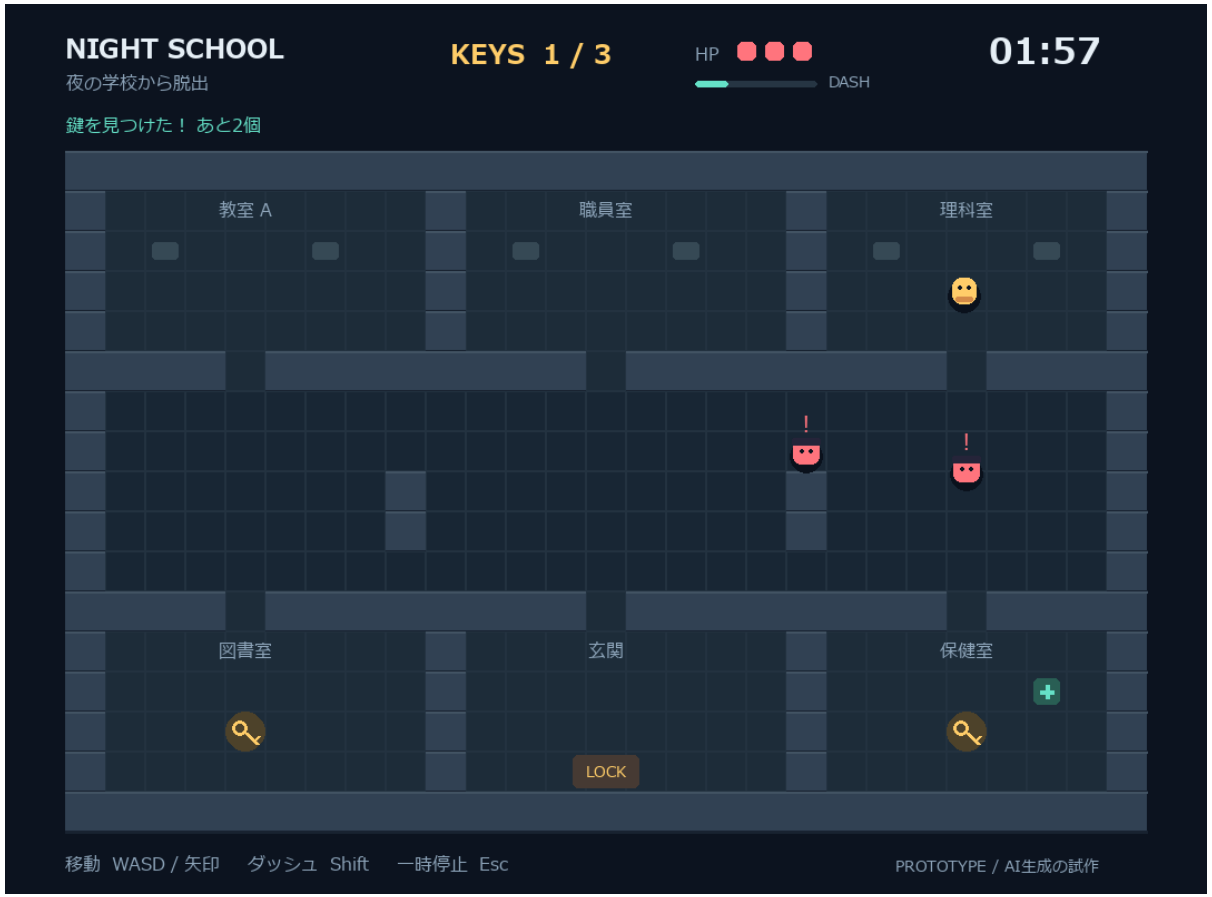
自主教材 / 2026.11.03 / 担当・仕様はグループ相談で確定

今回の到達目標

- 全員が共通ベースを起動し、担当の入口を説明できる。
- 5つの機能を、共通のデータと呼び出し方で作る。
- 各担当で確認してから順番に統合し、完成まで遊べる形にする。

読む順序：このスライド → 共通演習 → 自分の担当演習

共通ベースで、すでに遊べる



- 移動・ダッシュ・HP
- 鍵3個を集めて出口へ
- 敵の巡回と単純追跡
- 120秒以内に脱出

この試作品を起点に、新しい機能を追加する。

5人の担当案

- 代表者 kusupon：マップ・2ステージ・配置検証・統合
- あなた c0a25107a3：敵の探索・追跡AI
- メンバーA：隠れる・足音
- メンバーB：色付き鍵・扉・スイッチ
- メンバーC：ミッション・得点・ランキング

残り3人の名前を決め、担当の境界を全員で確認する。

最初の話し合いで決めること

- 完成の最低条件：2ステージ、脱出、失敗、再挑戦。
- 操作：矢印・Shift・Eで隠れる・Fでスイッチ。
- 担当名、共通仕様、統合の順番、確認する日時。

追加アイデアは、必須機能がつながってから。

ゲーム完成までの順序

- 1 相談：担当・最低完成条件・APIを合意
- 2 共通準備：contracts / Worldの枠、全員の起動確認
- 3 個別実装：担当演習を小さな段階で進める
- 4 統合：1つのPRごとに起動と基本操作を確認
- 5 完成確認：全員で通しプレイ、説明と提出物を仕上げる

第5回は個人実装を進め、第6回の統合につなげる。

コードは担当ごとに分ける

- game.py：イベント・更新順・画面遷移・接続
- world.py / stages.py：地形・座標・経路・視線・扉
- player.py / enemy_ai.py：主人公の操作 / 敵の状態と移動
- gimmicks.py / missions.py：仕掛け / 結果と保存
- settings.py / contracts.py：定数 / 全員が使うデータ型

各機能からgame.pyをimportしない。

座標と時間の約束

- Cellは (列, 行)。配列は grid[行][列]。
- 画面上のpositionはVector2、距離はピクセル。
- dtは秒。移動量は speed × dt、タイマーはdtだけ減らす。

world.center((3, 1)) → マス中心の画面座標
world.cell_at(position) → (列, 行)

座標の変換はWorldにまとめる。

1フレームの処理をそろえる

- 1 主人公：移動・スタミナ・足音を更新
- 2 仕掛け：鍵取得・扉の開放・取得をミッションへ通知
- 3 敵：視認・足音・状態・移動を更新
- 4 接触：被弾したときだけミッションへ通知
- 5 結果：HP → 時間切れ → 脱出の順で判定

描画では状態や保存データを変更しない。

演習01：Worldと2ステージ

- 固定マップをStageへ移し、Worldから読む。
- 通行・壁・経路・視線に、同じ扉の状態を反映する。
- 鍵とスイッチを順番に使い、出口まで届くか検証する。

確認：扉の手前に、必要な鍵がある。

演習02：敵は3つの状態を持つ

- PATROL / 巡回：決めた経路を歩く
- CHASE / 追跡：見えている主人公を追う
- SEARCH / 捜索：最後に見た場所や、音の位置を探す

発見 → CHASE　　見失う → SEARCH
捜索が終わる → PATROL

見えていない主人公の現在位置を追い続けない。

敵AI：判断する順序

- stunnedなら停止する。
- 見えていればCHASE。最後に見た位置を更新する。
- 見えずに音が聞こえたら、その発生位置を捜索する。
- 何もなければ、今の巡回または捜索を続ける。

捜索タイマーを毎フレーム初期化しない。

演習03：隠れる・足音

- 隠れ場所の近くでEを押すと、hiddenを切り替える。
- 隠れている間は移動・取得・視認・接触被弾を止める。
- 実際にダッシュ移動したときだけ、足音を発生させる。

NoiseEvent(position=(x, y), radius=240)
全敵へ同じtupleを渡し、そのフレームで破棄

壁に押しつけているだけなら、足音を出さない。

演習04：色付き鍵・扉・スイッチ

- inventoryへ鍵IDを追加する。取得は1回だけ。
- 対応する鍵で扉を開き、通行・視線に即反映する。
- Fで近くのスイッチを1回だけ動かす。
- 必須鍵がそろい、出口へ着いたらクリア。

鍵を扉の奥へ置くと、進行不能になる。

演習05：結果と得点を作る

- クリア基本点：300点
- 残り時間：整数部分 × 10点
- 残りHP / 取得鍵：HP × 100点 / 鍵 × 50点
- ボーナス：無被弾 +200点 / 残り60秒以上 +100点

残り90秒・HP3・鍵3・無被弾 → 1,950点

失敗は0点。結果の確定は1プレイ1回。

ランキングは壊れても遊べる設計に

- saves/ranking.jsonへクリア結果の上位5件を保存する。
- 初回でファイルがないときは、空の記録として開始する。
- 壊れたJSONや保存失敗があっても、ゲームを止めない。

保存は結果確定時に1回。drawから保存しない。

相手の完成を待たずに試す

- starter/contracts.pyでデータの名前をそろえる。
- starter/sandbox.pyで小さなWorldと仮Playerを使う。
- 敵AIなら「見える→隠れる→捜索」を順番に確認する。

sandboxは接続練習用。本番との統合確認も必要。

各担当が先に確認すること

- World：範囲外、閉じた扉、進行不能の検出
- 敵AI：壁越しに見えない、捜索終了、巡回復帰
- 主人公：隠れる、出る、足音、壁に押しつけた移動
- 仕掛け：二重取得、二重操作、鍵不足の出口
- ミッション：59.9秒と60秒、壊れた記録、再挑戦

動いた例に加えて、動かないべき例も確認する。

GitHubへは担当ブランチから

- 代表者リポジトリをForkし、自分の機能ブランチで作業。
- 小さな区切りでcommitし、自分のoriginへpush。
- 代表者のmain宛てにPRを出し、変更点と確認結果を書く。

あなたのブランチ：C0A25107/enemy-ai

mainへ取り込む前に、代表者と動作を確認する。

統合は1つずつ進める

- 1つ取り込むたびに起動し、基本操作を確認する。
- 引数の変更は、呼び出し元と一緒に直す。
- 再挑戦で鍵・扉・敵・得点・保存フラグを初期化する。

World → 仕掛け → 主人公 → 敵AI → ミッション

不具合が出たら、直前に取り込んだ変更から調べる。

完成条件とREADME

- 両ステージで脱出でき、失敗・一時停止・再挑戦が動く。
- ほかの人がREADMEだけで起動・操作できる。
- 担当、追加機能、参考にしたベース、確認方法を記載する。

提出方法・採点基準は、教員の講義資料を優先する。

いま取り掛かること

- 全員：共通演習を読み、担当名と仕様を確定する
- 代表者：contractsとWorldの準備を共有する
- あなた：演習02の状態定義とcan_seeから始める
- 各担当：最初の動作確認を終え、小さくcommitする

詰まったら「条件・期待した動作・実際の動作」を共有する。